

atm_machine.py

```
1 # Simple ATM simulation
2 # This program stores balance + transaction history in a file
3 # and lets the user deposit, withdraw and print a small report.
4 # Created by: Maithah Almesmari (202305693)
5
6
7 # text files used to store account data and printed report
8 ACCOUNT_FILE = "project_account.txt"
9 REPORT_FILE = "Project_account_print.txt"
10
11 # fake account information (hardcoded for this project)
12 PASSWORD = "werdf12345we"
13 OWNER_NAME = "Maithah Almesmari"
14 ACCOUNT_NUMBER = "202305693"
15
16 # program limits and rules
17 # (kept as constants so they are easy to change later)
18 MAX_HISTORY = 6
19 MAX_PRINT = 5
20 DEFAULT_BALANCE = 15000
21
22
23 MAX_PASSWORD_TRIES = 3
24 MAX_MENU_TRIES = 4
25 MAX_PRINT_TX = 5
26 LOW_BALANCE_LIMIT = 3000
27 LOW_BALANCE_FEE = 2
28
29
30
31
32 # Loads account info from file.
33 # If file doesn't exist yet, we create it with a default balance.
34 def load_account():
35
36     import os
```

```
37 | # read file line by line and detect which section we are in
38 | if not os.path.exists(ACCOUNT_FILE):
39 |     balance_list = [DEFAULT_BALANCE]
40 |     trans_list = []
41 |     save_account(balance_list, trans_list)
42 |     return balance_list, trans_list
43 |
44 | balance_list = []
45 | trans_list = []
46 | section = None
47 |
48 | infile = open(ACCOUNT_FILE, "r")
49 | line = infile.readline()
50 | while line != "":
51 |     line = line.rstrip()
52 |     if line == "[BALANCE_LIST]":
53 |         section = "balance_list"
54 |     elif line == "[TRANS_LIST]":
55 |         section = "trans_list"
56 |     elif section == "balance_list" and line != "":
57 |         balance_list.append(float(line))
58 |     elif section == "trans_list" and line != "":
59 |         parts = line.split(",")
60 |         if len(parts) == 2:
61 |             trans_list.append((float(parts[0]), parts[1]))
62 |         line = infile.readline()
63 | infile.close()
64 | return balance_list, trans_list
65 |
66 |
67 | # Saves balances and transaction history back to the file.
68 | # This overwrites the file each time the account changes.
69 | def save_account(balance_list, trans_list):
70 |
71 |     outfile = open(ACCOUNT_FILE, "w")
72 |     outfile.write("[BALANCE_LIST]\n")
73 |     for b in balance_list:
74 |         outfile.write(str(b) + "\n")
```

```
75 | outfile.write("[TRANS_LIST]\n")
76 | for amount, t_type in trans_list:
77 |     outfile.write(str(amount) + "," + t_type + "\n")
78 | outfile.close()
79 |
80 |
81 |
82 |
83 |
84 | # Returns the most recent balance (last value in the list)
85 | def check_balance(balance_list):
86 |
87 |     return balance_list[-1]
88 |
89 |
90 |
91 |
92 |
93 | # Handles deposit operation and updates file + history
94 | def add_to_account(balance_list, trans_list):
95 |
96 |     while True:
97 |         # blank input means user wants to cancel and go back
98 |         raw = input("\n Enter amount to deposit (AED): ").strip()
99 |
100 |         if raw == "":
101 |             print(" Returning to menu.")
102 |             return balance_list, trans_list
103 |
104 |         amount = float(raw)
105 |
106 |         if amount <= 0:
107 |             print(" Amount must be positive. Try again.")
108 |             continue
109 |
110 |         new_balance = check_balance(balance_list) + amount
111 |         balance_list.append(new_balance)
112 |         if len(balance_list) > MAX_HISTORY:
```

```

113     balance_list.pop(0)
114
115     trans_list.append((amount, "CR"))
116     save_account(balance_list, trans_list)
117     print(" Deposit successful. New balance: %.2f AED" % new_balance)
118     return balance_list, trans_list
119
120
121
122
123 # Handles withdrawal and low-balance fee logic
124 def withdraw_from_account(balance_list, trans_list):
125
126     current = check_balance(balance_list)
127
128     while True:
129         raw = input("\n Enter amount to withdraw (AED): ").strip()
130
131         if raw == "":
132             print(" Returning to menu.")
133             return balance_list, trans_list
134
135         amount = float(raw)
136
137         if amount <= 0:
138             print(" Amount must be positive. Try again.")
139             continue
140
141         if amount > current:
142             print(" You can't withdraw more than your balance, please try again.")
143             continue
144
145         new_balance = current - amount
146         # apply small fee if balance drops below allowed limit
147         if new_balance < LOW_BALANCE_LIMIT:
148             new_balance = new_balance - LOW_BALANCE_FEE
149         print(" Note: Balance below 3,000 AED  AED %d fee deducted." % LOW_BALA
NCE_FEE)

```

```
150
151     balance_list.append(new_balance)
152     if len(balance_list) > MAX_HISTORY:
153         balance_list.pop(0)
154
155     trans_list.append((amount, "DP"))
156     save_account(balance_list, trans_list)
157     print(" Withdrawal successful. New balance: %.2f AED" % new_balance)
158     return balance_list, trans_list
159
160
161
162
163
164 # Creates a simple text report and saves it to a file
165 def print_output(balance_list, trans_list):
166
167     while True:
168         raw = input("\n How many recent transactions to print? (1-%d): " % MAX_PRINT_TX)
169         X.strip()
170
171         if raw == "":
172             print(" Returning to menu.")
173             return
174
175         n = int(raw)
176
177         if n < 1 or n > MAX_PRINT_TX:
178             print(" Please enter a number between 1 and %d." % MAX_PRINT_TX)
179             continue
180
181         if len(trans_list) >= n:
182             last_transactions = trans_list[-n:]
183         else:
184             last_transactions = trans_list
185
186         max_bal = max(balance_list)
```

```

187 | min_bal = min(balance_list)
188 |
189 | deposits = []
190 | withdrawals = []
191 | for amount, t_type in trans_list:
192 |     if t_type == "CR":
193 |         deposits.append(amount)
194 |     else:
195 |         withdrawals.append(amount)
196 |
197 |
198 | if len(deposits) > 0:
199 |     max_cr = max(deposits)
200 |     min_cr = min(deposits)
201 | else:
202 |     max_cr = 0
203 |     min_cr = 0
204 |
205 | if len(withdrawals) > 0:
206 |     max_dp = max(withdrawals)
207 |     min_dp = min(withdrawals)
208 | else:
209 |     max_dp = 0
210 |     min_dp = 0
211 |
212 |
213 | sep = "*" * 56
214 | lines = []
215 |
216 |
217 | lines.append("Name:%s: Account number %s" % (OWNER_NAME, ACCOUNT_NUM-
MBER))
218 | lines.append("Account: value")
219 |
220 | if len(balance_list) >= MAX_PRINT:
221 |     recent_bal = balance_list[-MAX_PRINT:]
222 | else:
223 |     recent_bal = balance_list

```

```

224
225     for i in range(1, MAX_PRINT + 1):
226         if i <= len(recent_bal):
227             val = recent_bal[i - 1]
228             if val == int(val):
229                 lines.append("%d-value%d    %d" % (i, i, int(val)))
230             else:
231                 lines.append("%d-value%d    %.2f" % (i, i, val))
232         else:
233             lines.append("%d-" % i)
234
235     lines.append("")
236     lines.append("Value:  Type")
237     for amount, t_type in last_transactions:
238         if amount == int(amount):
239             lines.append("%d:    %s" % (int(amount), t_type))
240         else:
241             lines.append("%.2f:    %s" % (amount, t_type))
242
243     lines.append(sep)
244     lines.append("Max value in account:    %g" % max_bal)
245     lines.append("Min value in account:    %g" % min_bal)
246     lines.append("Max CR in account:    %g" % max_cr)
247     lines.append("Min CR in account:    %g" % min_cr)
248     lines.append("Max DP in account:    %g" % max_dp)
249     lines.append("Min DP in account:    %g" % min_dp)
250     lines.append(sep)
251
252
253     report = ""
254     for line in lines:
255         report = report + line + "\n"
256
257
258     outfile = open(REPORT_FILE, "w")
259     outfile.write(report)
260     outfile.close()
261

```

```
262     print("\n" + report)
263     print(" Report saved to '%s'." % REPORT_FILE)
264     return
265
266
267
268
269 # Displays the ATM menu
270 def show_menu():
271
272     print("\n" + "-" * 50)
273     print(" Select an option:")
274     print(" 1. Check Balance")
275     print(" 2. Deposit (Add to Account)")
276     print(" 3. Withdraw from Account")
277     print(" 4. Print Output")
278     print(" Q. Quit")
279     print("-" * 50)
280
281
282
283
284
285 # Displays the ATM menu
286 def check_password_input():
287
288     print("\n" + "=" * 50)
289     print(" Welcome to the ATM")
290     print("=" * 50)
291
292     tries = 0
293     while tries < MAX_PASSWORD_TRIES:
294         entered = input("\nEnter password: ")
295
296         if entered == PASSWORD:
297             print("\nPassword accepted. Welcome!\n")
298             return True
299
```



```
300     tries = tries + 1
301     remaining = MAX_PASSWORD_TRIES - tries
302     if remaining > 0:
303         print("Incorrect password. %d attempt(s) remaining." % remaining)
304
305
306     print("\nYour account is locked, please contact your bank.")
307     return False
308
309
310
311
312 # Displays the ATM menu
313 def give_option(balance_list, trans_list):
314
315     wrong = 0
316
317     while True:
318         show_menu()
319         choice = input(" Your choice: ").strip()
320
321         if choice == "Q" or choice == "q":
322             print("\n Thank you for using our ATM. Goodbye!\n")
323             return
324
325         elif choice == "1":
326             wrong = 0
327             bal = check_balance(balance_list)
328             print("\n Current Balance: %.2f AED" % bal)
329
330         elif choice == "2":
331             wrong = 0
332             balance_list, trans_list = add_to_account(balance_list, trans_list)
333
334         elif choice == "3":
335             wrong = 0
336             balance_list, trans_list = withdraw_from_account(balance_list, trans_list)
337
```

```
338     elif choice == "4":
339         wrong = 0
340         print_output(balance_list, trans_list)
341
342     else:
343         wrong = wrong + 1
344         if wrong >= MAX_MENU_TRIES:
345             print("\n You didn't select the right choice. Exiting.\n")
346             return
347         remaining = MAX_MENU_TRIES - wrong
348         print(" Invalid choice. Enter 1-4 or Q. (%d attempt(s) left)" % remaining)
349
350
351 # Program starting point
352 def main():
353
354     balance_list, trans_list = load_account()
355
356     if check_password_input():
357         give_option(balance_list, trans_list)
358
359
360
361 main()
362
```